MiniVerse

ONE APP. ENDLESS MINI ADVENTURES.

Complete explainer: UI design, how every game works, offline same-device + local multiplayer, engine, coins, and the rules that keep the app fast and crash-free.

Android | Kotlin | Jetpack Compose | Room | Offline-first

Powered by Devanshu

How to use this document

This PDF merges the three MiniVerse blueprints (Game Hub Blueprint, Game Logic and Coin Economy, App Architecture) into one beginner-friendly guide. Read it top to bottom once, then use it as a checklist while you build. Every section answers one question:

Part	Question it answers
1. Big picture	What is MiniVerse and what are the final decisions?
2. UI design	What screens do we need, what do they look like, how does the user move between them?
3. Game catalogue	Which 28 games, how many players, which build phase?
4. How every game works	Rules, state, controls on one device, sync on two phones, result - for all 28 games.
5. Same-device multiplayer	How 2-4 people share one phone without conflicts.
6. Offline phone-to-phone multiplayer	How phones connect with no internet and stay in sync.
7. Game engine	The one reusable engine every game plugs into.
8. Match lifecycle	The state machine that prevents bugs.
9. MiniCoins economy	Rewards, wallet, shop, duplicate-payout protection.
10. Data and storage	What goes in RAM, Room, DataStore, cache.
11. No lag, no crash	Performance rules, error handling, memory and battery rules.
12. Project structure and tech stack	Folders, modules, libraries.
13. Testing	What to test before every release.
14. Build order	Step-by-step roadmap, week by week.
15. Decisions to confirm	The 4 choices to finish before coding.

Golden rules (remember these 6): 1) UI never decides rules, scores or coins. 2) Active game lives in RAM; Room stores only results and checkpoints. 3) One host decides everything in multiplayer. 4) Coins are saved first, animated second. 5) Fixed time step for physics. 6) Build one complete game (Tic Tac Toe) end to end before building the rest.

1. The big picture

MiniVerse is one lightweight Android app that contains many mini-games. Friends open it, pick a game and play together on the same phone, or connect two or more phones with no internet. It needs no account, no server and no subscription for the core experience. Each game is an independent module that plugs into a shared platform (engine, wallet, multiplayer, animations, storage). Adding game number 29 should be far easier than building game number 1.

Decision	Final choice
App name / tagline	MiniVerse - One App. Endless Mini Adventures. (Powered by Devanshu)
Platform / language	Android, Kotlin
UI	Jetpack Compose + Material 3
Architecture	Modular Clean Architecture + MVVM
Storage	Room (records), DataStore (settings), RAM + disk cache
Multiplayer	Same device (pass and play) + local phone-to-phone (Wi-Fi Direct / Bluetooth / optional Nearby Connections)
Network	Not required for any core gameplay
Games	28 planned; first release = 8 polished games
Currency	MiniCoins (MC) - cosmetic only, no cash value
Size goal	Under 25 MB (target, measure it)
Minimum Android	API 24 (Android 7.0) baseline, verify libraries
Monetisation	Optional later: ads / premium themes

Main objectives

- Launch games quickly with almost no loading.
- Work 100% offline, including multiplayer.
- Support 1-4 players on one device where the rules allow.
- Support multiplayer between phones using local connectivity.
- Save progress, achievements and scores locally.
- Keep every game independent so new ones can be added without touching the rest.
- Keep APK size and memory under control; run on 2 GB budget phones and flagships.

Honest note: no architecture can promise zero bugs. This design reduces lag and crashes through validation, automated tests, safe database transactions, bounded memory and graceful recovery - and then you must measure on real devices.

2. UI design - what screens we need

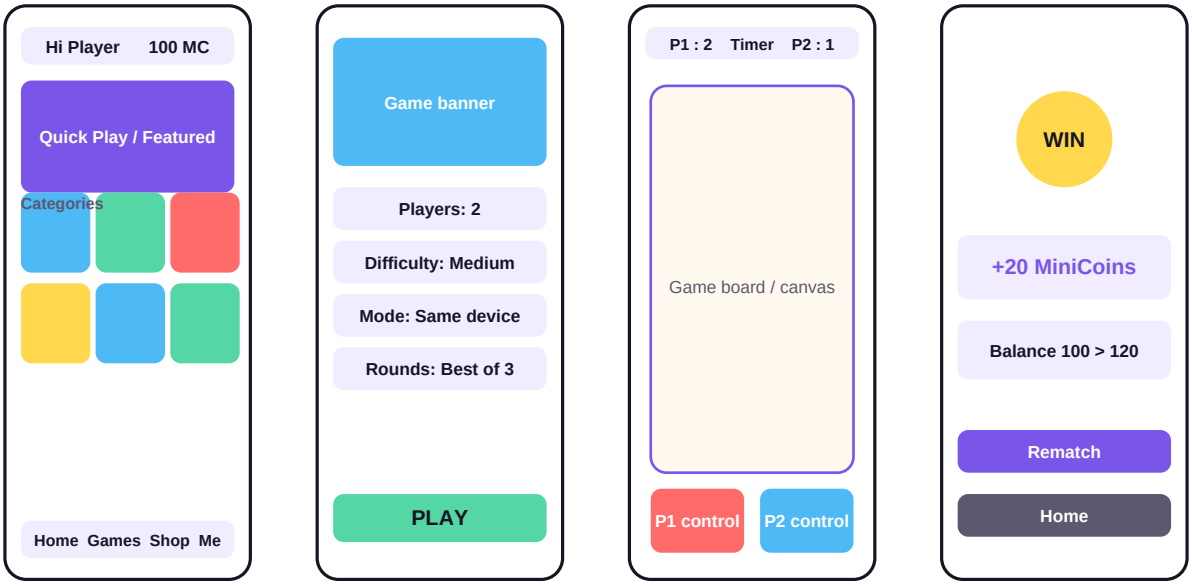
The app should feel like a colourful arcade world: rounded cards, big thumb-friendly buttons, bright accent colours and short, snappy animations (under 300 ms for transitions).

2.1 Colour palette

Role	Colour	Use
Primary purple	#7955E8	Top bars, primary buttons, headings
Bright yellow	#FFD84D	Coins, highlights, rewards
Coral	#FF6B6B	Player 1 colour, warnings, loss
Sky blue	#4DB9F5	Player 2 colour, info
Mint green	#55D6A5	Success, win, play buttons
Background (light)	#FFF9F0	Light theme background
Background (dark)	#171523	Dark theme background

Typography: use the system font (small APK). Support font scaling and keep minimum touch targets of 48 dp.

2.2 Core screens at a glance



1. Home | 2. Game details / setup | 3. Gameplay | 4. Result + reward

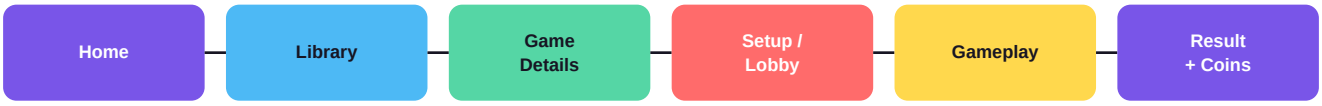
2.3 Full screen list and what each must contain

Screen	Must contain	Main data/logic
Splash	Logo, tagline, "Powered by Devanshu". Loads theme, Room, game registry, profile in the background (target under 2 s).	App initialisation only
Onboarding (first run)	Choose nickname, choose avatar. Gives starting coins (recommended 100 MC).	PlayerEntity + wallet created once
Home dashboard	Coin balance, Quick Play, Featured games, Categories, Recently played, Daily challenge, bottom navigation.	HomeUiState: games, recents, wallet

Screen	Must contain	Main data/logic
Game library	Grid of all games, category filter chips, search, favourites, player-count badges.	Lazy grid, registry list
Game details	Banner, rules text, player range, difficulty, play mode (Same device / Create room / Join room), Start.	GameDefinition
Match setup / Lobby	Player names + avatars, seat colours, rounds (single / best of 3 / 5), difficulty. For multiplayer: room code, connected devices, Ready buttons.	Session + player list + ready states
Countdown	3-2-1-GO overlay.	Match state COUNTDOWN
Gameplay	Top bar (scores, timer, pause), game canvas/board, per-player controls.	Engine state rendered; input -> action
Pause menu	Resume, restart, settings (sound/vibration), quit (confirm: no coins on quit).	PAUSED state
Result + reward	Winner/Draw/Good try animation, coin reward, balance counter, Rematch, Home.	Reward saved BEFORE animation
Wallet	Balance, transaction history list.	Ledger from Room
Shop	Avatars, game backgrounds, board themes, profile frames, emote packs (cosmetic only).	Purchase use case (atomic)
Achievements	Trophies, badges, streaks, progress bars.	AchievementEntity
Profile + History	Nickname, avatar, wins, matches, personal bests, match history.	Match + stats queries
Settings	Sound, vibration, theme (light/dark/dynamic), reduced motion, language, accessibility, clear data.	DataStore

2.4 Navigation structure

```
Splash -> Onboarding (first run only) -> Home
Home -> Game Library -> Game Details -> Match Setup
Match Setup -> [Same Device] -> Countdown -> Gameplay -> Result -> (Rematch | Home)
Match Setup -> [Create Room | Join Room] -> Lobby -> Countdown -> Gameplay -> Result
Home -> Wallet | Shop | Achievements | Profile | History | Settings
```



2.5 Adaptive layout (works on every phone)

- Small phones: 2-column game cards, compact controls. Large phones: 2-3 columns. Tablets: 3-4 columns, wider boards. Landscape: bigger canvas with dedicated control zones.
- Use WindowSizeClass / adaptive APIs; use BoxWithConstraints only when needed.
- Never use fixed-width boards - use aspect ratios (a board is always square or a fixed ratio, scaled to fit).
- Respect status bar, navigation bar and display cutouts (insets).
- Physics games use a **virtual world** (e.g. 1000 x 600 units). The renderer scales the world to any screen, so a 360x800 phone and a 1280x800 tablet play with identical rules.
- Provide reduced-motion option: disables confetti and strong shakes. Also allow muting sound and vibration.

2.6 Reward screen sequences

Result	Sequence	Timing
Win	Freeze game, dim background, trophy enters, confetti, "YOU WON!", coins fly to balance, counter rolls old to new balance, summary "+20 MiniCoins".	Transition 300 ms, trophy 500 ms, confetti 1500 ms, coin flight 800 ms, counter 700 ms. Whole screen 2-3 s.
Draw	Both avatars appear, handshake animation, "IT'S A DRAW!", small coin particles, both +10.	Same timings, lighter effects
Loss	Result shown, funny disappointed character, "GOOD TRY!", +5 participation coins, Rematch button appears.	Keep it kind and short

Use Compose animations and light vector/sprite assets. Avoid video files or heavy animation libraries. Important: the animation only *shows* the reward that is already saved - it never decides it.

3. The 28-game catalogue

Five categories. Some games are true simultaneous multiplayer, some are turn-based, and some are score challenges. We do not force single-player arcade games into multiplayer.

A. Board and classic

#	Game	Players	Type	Build phase
1	Mini Ludo	2-4	Turn-based, dice	Phase 1
2	Snakes & Ladders	2-4	Turn-based, dice	Phase 2
3	Tic Tac Toe	2	Turn-based	Phase 1
4	Connect Four	2	Turn-based	Phase 1
5	Dots & Boxes	2	Turn-based	Phase 1
6	Mini Chess	2	Turn-based, strategy	Phase 2

B. Party and multiplayer

#	Game	Players	Type	Build phase
7	Rock Paper Scissors	2	Simultaneous, party	Phase 3
8	Reaction Battle	2-4	Real-time, party	Phase 3
9	Finger Football	2	Real-time, physics	Phase 3
10	Mini Sumo	2	Real-time, physics	Phase 3
11	Ping Pong Battle	2	Real-time, physics	Phase 4
12	Guess The Word	2-4	Turn-based, party	Phase 5

C. Arcade and physics

#	Game	Players	Type	Build phase
13	Pinball Master	1-2	Arcade, physics	Phase 4
14	Tank Battle	2	Real-time, arcade	Phase 4
15	Crazy Runner	1	Endless arcade	Phase 5
16	Sky Jumper	1	Endless arcade	Phase 5
17	Whack-a-Mole	1-2	Reaction, party	Phase 5
18	Air Hockey	2	Real-time, physics	Phase 4

D. Brain and puzzle

#	Game	Players	Type	Build phase
19	Memory Match	1-2	Puzzle	Phase 1
20	Number Merge	1	Puzzle (2048 style)	Phase 6
21	Word Scramble	1-2	Puzzle, word	Phase 6
22	Quick Math	1-2	Puzzle, speed	Phase 6
23	Simon Says	1-2	Memory, rhythm	Phase 7
24	Color Panic	1	Reaction, arcade	Phase 7

E. Crazy challenges

#	Game	Players	Type	Build phase
25	Fall Challenge	1-4	Party, platform	Phase 7
26	Bomb Squad	2-4	Party, pass the bomb	Phase 7
27	Balloon Pop	1-2	Reaction, arcade	Phase 5
28	Maze Escape	1-2	Puzzle, race	Phase 7

Suggested first release (8 polished games): Tic Tac Toe, Connect Four, Snakes & Ladders, Memory Match, Reaction Battle, Rock Paper Scissors, Air Hockey, Pinball Master. This covers board, puzzle, reaction and arcade types, so it proves the whole architecture.

4. How every game works

Each card below tells you: the rules, the data you store, how one phone plays it, how two phones sync it, how the reward is decided, and what to watch out for so it does not lag or crash. Build games in the order of the phases in section 14.

Three families, three ways of syncing

Family	Examples	How state moves between phones
Turn-based	Ludo, Snakes and Ladders, Tic Tac Toe, Connect Four, Dots and Boxes, Chess, Memory, Word games	Guest sends a tiny action. Host validates and broadcasts the accepted event. Few messages, very stable.
Real-time physics	Air Hockey, Ping Pong, Finger Football, Tank, Sumo, Fall Challenge	Send only inputs. Host simulates on a fixed 60 Hz step and sends snapshots about 20 times per second. Guests interpolate.
Score challenge	Pinball, Runner, Sky Jumper, Number Merge, Color Panic	Each phone plays alone. Compare final scores at the end. No live sync needed - the easiest and safest.

01. Mini Ludo	2-4 players | Turn-based, dice
How it plays	Each player has 4 tokens. Roll the dice; a 6 releases a token from base. Move tokens along a fixed 52-cell route, land on an opponent (not on a safe cell) to send it back to base, then enter the home column. First player with all 4 tokens home wins. A 6 gives an extra roll.
State + algorithm	State: route index per token (-1 base, 0-51 track, 52-57 home column), current seat, last dice, consecutive-six count. Route is a constant array - never pixel based. Dice rolled by the host/engine only.
One device	Pass and play. Board rotates or the active seat is highlighted; big Roll button and tappable tokens. Only legal tokens glow.
Two phones	Guest sends SELECT_TOKEN(index). Host validates turn + dice + legality, then broadcasts the accepted event with new state.
Result / coins	Rank by tokens home. 1st = Win, others = Loss reward. Quit = Abandoned (0).
Avoid lag/crash	Precompute the route array once. Animate token with Compose animateFloat, not a game loop. Reject a move if dice is not rolled yet.

02. Snakes & Ladders	2-4 players | Turn-based, dice
How it plays	Board 1-100. Roll dice, move forward. Land on a ladder bottom to climb, on a snake head to slide down. Must roll exact number to land on 100 (optional rule: overshoot = stay).
State + algorithm	State: position per player, turn, map of jumps {4:14, 9:31, 17:7 ...}. Algorithm: newPos = pos + dice; if > 100 stay; then apply jump map once.
One device	Pass and play, one big Roll button, pawn hops cell by cell.
Two phones	Guest only asks ROLL; host rolls and broadcasts (dice, newPos). Guests never roll themselves.
Result / coins	First to 100 wins; others Loss reward.
Avoid lag/crash	Jump map is a constant. Apply jump only once (no chain loops). Cap hop animation to 600 ms.

03. Tic Tac Toe	2 players | Turn-based
How it plays	3x3 grid. Players alternate placing X and O. First to get 3 in a row/column/diagonal wins; full board without a line = draw.
State + algorithm	State: board[9] (0 empty,1 X,2 O), currentPlayer, status. Check the 8 winning lines after every move.
One device	Pass and play: tap a cell. Show whose turn at top. Vertical-split names for facing players.
Two phones	Guest sends PlaceMark(cell). Host checks turn + empty cell, broadcasts.
Result / coins	Win 20 / Draw 10 / Loss 5. Best of 3 / 5 supported.
Avoid lag/crash	Use enums (not strings) for status. Ignore double taps with a 150 ms input lock.

04. Connect Four	2 players | Turn-based
How it plays	7 columns x 6 rows. Tap a column; the disc drops to the lowest empty row. First to 4 in a row (horizontal, vertical, diagonal) wins. Full board = draw.
State + algorithm	State: grid[6][7], current player. After a drop check 4 directions from the new disc only (cheap).
One device	Pass and play: tap a column; falling disc animation 300 ms.
Two phones	Guest sends DropDisc(column). Host validates column not full.
Result / coins	Win 20 / Draw 10 / Loss 5.
Avoid lag/crash	Check only lines through the last disc. Block input while disc is animating.

05. Dots & Boxes	2 players | Turn-based
How it plays	Grid of dots. On your turn draw one line between adjacent dots. Completing the 4th side of a box scores 1 point and gives another turn. Most boxes when all lines are drawn wins.
State + algorithm	State: horizontal/vertical edge flags, box owner array, scores. After a line check the (max 2) adjacent boxes.
One device	Pass and play: tap an edge; completed box fills with player colour.
Two phones	Guest sends DrawEdge(id). Host replies with new edges + owners + whose turn.
Result / coins	Higher boxes wins; equal = draw.
Avoid lag/crash	Use bit sets for edges - tiny memory. Make edge touch area at least 48dp.

06. Mini Chess	2 players | Turn-based, strategy
How it plays	Small board (5x5 or 6x6 mini-chess) with standard piece moves. Capture the opposing king (checkmate) to win. Draw by stalemate or repetition.
State + algorithm	State: board with piece codes, turn, castling disabled for mini variant. Legal-move generator per piece; filter moves that leave own king in check.
One device	Pass and play: tap piece (legal squares highlight), tap target. Board flips optional.
Two phones	Guest sends Move(from,to). Host validates against its own legal-move list.
Result / coins	Win/Draw/Loss reward. Optional timer per move.
Avoid lag/crash	Generate moves lazily only for the selected piece. Keep AI (if added later) on a background dispatcher with depth limit.

07. Rock Paper Scissors	2 players | Simultaneous, party
How it plays	Both players secretly choose Rock, Paper or Scissors, then reveal together. Rock beats Scissors, Scissors beats Paper, Paper beats Rock. First to 2 (best of 3) or 3 (best of 5) wins.
State + algorithm	State: round, choices (hidden until both locked), scores. Result table 3x3.
One device	Shared screen split in two halves: each player taps in own half; a cover hides the choice until both confirm.
Two phones	Each device sends a LOCKED hash/ack, then host reveals both choices together - no one can peek.
Result / coins	Match winner Win 20, loser 5. Draw round replays.
Avoid lag/crash	Never reveal a choice early. Timeout of 5 s auto-picks random.

08. Reaction Battle	2-4 players | Real-time, party
How it plays	Screen waits a random 1-5 s, then a target (colour flash / GO!) appears. First player to tap their zone scores. Tapping early gives a penalty. First to 5 points wins.
State + algorithm	Timing with SystemClock.elapsedRealtimeNanos(). Random delay chosen once per round. Reject taps before the target timestamp.
One device	Screen divided into 2 or 4 zones (one per player), each with a big tap pad.
Two phones	Host decides the target time; guests send tap timestamp offset; host compares using its clock after round-trip compensation.
Result / coins	Most points wins.
Avoid lag/crash	One timer job per round; cancel it on exit. Do not trust delay() as the clock.

09. Finger Football	2 players | Real-time, physics
How it plays	Top-down mini pitch. Each player flicks/drags a striker to hit the ball into the opponent goal. First to 5 goals or most goals in 2 minutes wins.
State + algorithm	Virtual world 1000x600. Circle-circle collisions, friction 0.98/step, fixed 1/60 s step, goal = event.
One device	Phone held between players; each controls their own half by dragging.
Two phones	Send input (drag vector + tick) only; host simulates and sends snapshots at 20 Hz; guests interpolate.
Result / coins	Higher goals wins; tie = draw (or golden goal).
Avoid lag/crash	Fixed timestep accumulator; cap max 5 steps per frame to avoid spiral of death.

10. Mini Sumo	2 players | Real-time, physics
How it plays	Two round fighters on a circular platform. Push the opponent out of the ring. Best of 3 rounds.
State + algorithm	Circle bodies with mass, velocity, dash with cooldown. Ring-out = centre distance > radius.
One device	Two thumb joysticks/dash buttons at the bottom corners.
Two phones	Host-authoritative physics; guests send joystick vector + dash; snapshot reconciliation.
Result / coins	Round wins decide match.
Avoid lag/crash	Limit dash to once per 1.5 s so input spam cannot flood network.

11. Ping Pong Battle	2 players | Real-time, physics
How it plays	Classic paddle and ball. Each paddle on one side, ball speeds up each hit. Miss = opponent scores. First to 7.
State + algorithm	Ball position/velocity, paddle clamp, reflection angle depends on hit offset. Fixed step 60 Hz.
One device	Portrait: paddles at top and bottom, each player drags own paddle with a finger.
Two phones	Send paddle X position (throttled 30/s). Host sends ball state snapshot.
Result / coins	First to 7 wins.
Avoid lag/crash	Reuse one Ball object; do not allocate per frame.

12. Guess The Word	2-4 players | Turn-based, party
How it plays	One player secretly types/selects a word (or picks from a local list) and others guess letters like hangman within 6 wrong guesses. Roles rotate each round.
State + algorithm	State: secret word, revealed mask, wrong letters, guesser seat. Word list stored as a bundled compressed text file.
One device	Pass the phone: setter enters word, then hides it before passing.
Two phones	Host stores the secret; guests send letter guesses; host replies with mask only (word never sent).
Result / coins	Most words guessed over rounds wins.
Avoid lag/crash	Load word list lazily from assets, not at startup.

13. Pinball Master	1-2 players | Arcade, physics
How it plays	Launch the ball with a plunger, control left/right flippers, hit bumpers and targets for points. Lose ball at the bottom drain; 3 balls per game. 2 players alternate balls.
State + algorithm	Ball (position, velocity, radius), flippers as rotating segments, bumpers apply impulse and emit score events.
One device	Left/right half of screen = flippers. Alternate turns on one device for 2 players.
Two phones	Score challenge: each phone plays its own table; host compares final scores (no live sync needed).
Result / coins	Higher score wins; single player uses Bronze/Silver/Gold targets.
Avoid lag/crash	Physics outside Compose recomposition; use Canvas/SurfaceView loop. Substeps for fast ball to avoid tunnelling.

14. Tank Battle	2 players | Real-time, arcade
How it plays	Two tanks on a small map with walls. Move and aim, shoot shells; a shell bounces once off walls. 3 hits kills a tank. Best of 3.
State + algorithm	Grid map, tank rect, bullet circle, ray-rect collision. Reload 0.8 s.
One device	Left side joystick + fire for P1 / right side for P2 (landscape).
Two phones	Send move/aim/fire inputs; host simulates and snapshots.
Result / coins	Last tank alive wins round.
Avoid lag/crash	Pool bullets (max 10 objects). Clamp tank speed.

15. Crazy Runner	1 players | Endless arcade
How it plays	Character runs automatically; tap to jump, swipe down to slide over obstacles. Speed increases over time. Score = distance + coins collected.
State + algorithm	Scrolling world with spawn table; difficulty curve = speed = base + 0.02*time; collision AABB.
One device	Single player; optional ghost score compare by passing phone.
Two phones	Score challenge only: each player plays, higher distance wins.
Result / coins	Bronze/Silver/Gold targets + personal-best bonus.
Avoid lag/crash	Object pool for obstacles; recycle off-screen items.

16. Sky Jumper	1 players | Endless arcade
How it plays	Character bounces on platforms upward. Tilt or drag to steer horizontally; avoid falling. Height = score.
State + algorithm	Constant gravity, bounce velocity on platform hit (only when falling), procedural platform generation above camera.
One device	Single player; pass-the-phone high-score duel.
Two phones	Score challenge.
Result / coins	Medal targets; personal best bonus.
Avoid lag/crash	Delete platforms below the camera; generate in chunks.

17. Whack-a-Mole	1-2 players | Reaction, party
How it plays	Moles pop up from a 3x3 grid for 0.6-1.2 s. Tap them to score; bombs/penalty moles lose points. 30-second round.
State + algorithm	Spawn scheduler with random hole, lifetime shrinks as time passes. Score events.
One device	1 player, or 2 players sharing screen (each owns a colour of mole).
Two phones	Host schedules moles with timestamps; guests send tap events.
Result / coins	Highest score in 30 s.
Avoid lag/crash	Single timer job; ensure two moles never use same hole.

18. Air Hockey	2 players | Real-time, physics
How it plays	Table with a puck and two mallets. Hit the puck into the opponent goal. First to 7 wins.
State + algorithm	Virtual world; mallet follows finger with max speed clamp; puck friction + wall reflection; goal events.
One device	Phone flat between players; each uses their half with a finger.
Two phones	Send mallet position 30/s; host simulates puck at 60 Hz and sends snapshots at 20 Hz; interpolate.
Result / coins	First to 7 wins.
Avoid lag/crash	Fixed timestep + interpolation hides network jitter.

19. Memory Match	1-2 players | Puzzle
How it plays	Grid of face-down cards (4x4). Flip two cards: if they match they stay open, otherwise flip back. 2 players alternate; a match gives another turn.
State + algorithm	Fisher-Yates shuffle, state: card id, matched flag, selected (max 2). Temporary RESOLVING state locks input.
One device	Pass and play; card flip animation 250 ms.
Two phones	Host shuffles and sends the layout once; guests send FlipCard(index).
Result / coins	Most pairs wins; single-player = fewest moves.
Avoid lag/crash	Block taps while two cards are resolving.

20. Number Merge	1 players | Puzzle (2048 style)
How it plays	4x4 grid. Swipe to slide all tiles; equal tiles merge into their sum. New tile (2 or 4) appears after every valid move. Game over when no move is possible.
State + algorithm	Grid scan per direction, merge once per tile per move. Score = sum of merges.
One device	Single player; optional score duel.
Two phones	Score challenge.
Result / coins	Medal targets by score.
Avoid lag/crash	Operate on IntArray(16), not objects; save checkpoint on each move to resume.

21. Word Scramble	1-2 players | Puzzle, word
How it plays	A word with shuffled letters is shown; type/tap letters to rebuild it before the timer ends. More solved words = more points.
State + algorithm	Randomised permutation (must differ from original), dictionary check from asset file.
One device	Race mode: both players use halves of the screen with their own letter tiles.
Two phones	Host picks words and timestamps; guests send answers.
Result / coins	Most solved in 60 s.
Avoid lag/crash	Hash set of words loaded lazily on a background thread.

22. Quick Math	1-2 players | Puzzle, speed
How it plays	Equations appear (e.g. 7 x 8 = ?) with answer options. Answer fastest; wrong answer = penalty. Difficulty sets operator range.
State + algorithm	Bounded random generator: easy 1-10 +/-, medium x, hard mixed. Distractors near the true answer.
One device	Split screen, each player has own answer buttons.
Two phones	Host generates question seed list; guests send answer + time.
Result / coins	Highest score in 60 s.
Avoid lag/crash	Generate questions from a seed so both phones match.

23. Simon Says	1-2 players | Memory, rhythm
How it plays	Four coloured pads flash a growing sequence. Repeat it in order. One wrong tap ends the turn.
State + algorithm	Sequence list; user input validated as prefix of sequence. Playback via a single coroutine.
One device	2 players alternate: longest sequence wins.
Two phones	Host owns the sequence; guests send tapped pad ids.
Result / coins	Longest sequence.
Avoid lag/crash	Cancel playback coroutine on leave; use sound pool, not MediaPlayer.

24. Color Panic	1 players | Reaction, arcade
How it plays	A colour name appears in a different ink colour. Tap the matching colour button within a shrinking time limit. 3 lives.
State + algorithm	Random word/ink pair, timer shrinks per level, combo multiplier.
One device	Single player.
Two phones	Score challenge.
Result / coins	Medal targets.
Avoid lag/crash	Pre-create the Paint/TextStyle objects once.

25. Fall Challenge	1-4 players | Party, platform
How it plays	Players guide characters through falling platforms / gaps; stay on the last stable platform. Platforms vanish after being touched.
State + algorithm	Simple tile map with decay timers; gravity + horizontal tap-zone control.
One device	Each player gets a screen zone with left/right tap area (up to 4).
Two phones	Host simulates; guests send left/right/jump input.
Result / coins	Last survivor wins.
Avoid lag/crash	Fixed map size; reuse tile objects.

26. Bomb Squad	2-4 players | Party, pass the bomb
How it plays	A bomb with hidden timer (10-30 s) is passed between players by tapping the next player. When it explodes the holder loses the round. Mini-tasks (tap fast, hold) may be required to pass it.
State + algorithm	Hidden timer value known only to host/engine; passes recorded as events.
One device	Pass the phone or tap player corner; screen shakes as timer shortens.
Two phones	Host holds the timer; guests only see 'tick' effect, never the real time (prevents cheating).
Result / coins	Last player without an explosion wins.
Avoid lag/crash	Vibration pattern uses VibratorManager with permission checks.

27. Balloon Pop

1-2 players | Reaction, arcade

How it plays	Balloons rise from the bottom; tap to pop for points. Golden = bonus, bomb balloon = penalty. 40-second round.
State + algorithm	Spawner with speed curve, circle hit-test, pooled balloons.
One device	1 player, or 2 players with split halves.
Two phones	Host spawns using a shared seed; guests send pop events.
Result / coins	Highest score.
Avoid lag/crash	Pool max ~25 balloons.

28. Maze Escape

1-2 players | Puzzle, race

How it plays	Reach the exit of a generated maze first (or alone before time ends). Control by swipe or on-screen D-pad.
State + algorithm	Maze generated by DFS / recursive backtracker; BFS used to guarantee a solvable path and for hints.
One device	Two players race in two identical mazes side by side.
Two phones	Host sends maze seed once; guests send their cell position changes.
Result / coins	Fastest to exit wins.
Avoid lag/crash	Generate maze on a background dispatcher; send only the seed.

5. Same-device multiplayer (offline, one phone)

One phone, 1-4 people, zero connectivity. The engine must not care whether a player is local or remote - the same `processAction(state, action, playerId)` is used. Only the *input source* changes.

5.1 Three same-device modes

Mode	Used by	How it works
Pass and Play (turn-based)	Ludo, Snakes and Ladders, Tic Tac Toe, Connect Four, Dots and Boxes, Chess, Guess the Word, Memory	Only the active player may act. A banner shows "Player 2 - your turn" with their colour. Others' taps are ignored by turn validation.
Split-screen / zones (real-time)	Reaction Battle, Air Hockey, Ping Pong, Whack-a-Mole, Balloon Pop, Word/Quick Math race	Screen is divided into zones, one per player. Each zone has its own touch area and score. Multi-touch is tracked per pointer id.
Shared-screen challenge	Fall Challenge, Bomb Squad, Finger Football	All players see one canvas; controls sit in separate corners (landscape recommended).

5.2 Seat layout for 2, 3 and 4 players

Players	Layout
2	Top/bottom halves (face-to-face) or left/right in landscape. Top half text is rotated 180 degrees for the opposite player.
3	Three zones: two at the bottom corners and one at the top (rotated).
4	Four corners; each name/score is rotated to face its player.

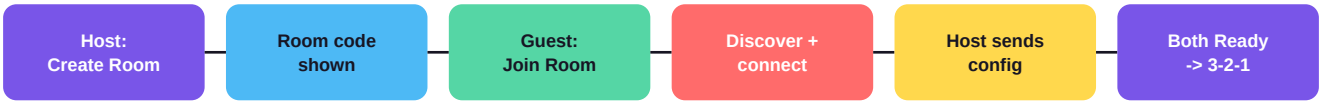
5.3 Rules that prevent bugs

- Each local player gets an id (`local-1`...`local-4`), a seat index and a colour that never changes during the match.
- Track touches by pointer id so two fingers in two zones never interfere; ignore a touch that starts in a zone but moves into another.
- Lock input during animations (disc falling, card flipping) to stop double moves.
- Screen must stay awake during a match (FLAG_KEEP_SCREEN_ON) and be released on exit.
- Pause automatically when the app goes to background; resume with a countdown.
- Hide secrets: Rock Paper Scissors and Guess the Word need a cover screen so the next player cannot see the previous choice.
- Best of 3 / 5 and tournaments are just a series of new sessions with a running series score.

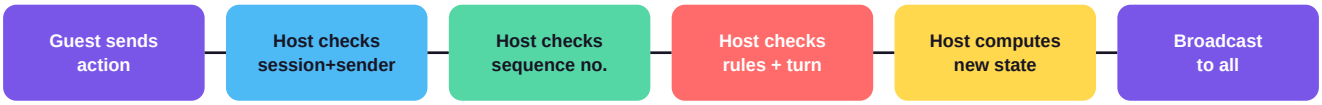
6. Phone-to-phone multiplayer with no internet

Two or more phones form a small local network. One phone is the **host** (the referee); the others are **guests**. The host holds the only true game state.

6.1 How a match starts



6.2 Host-authoritative loop (turn-based)



The host never trusts a guest's score or claim of winning; it calculates the result from accepted actions. A local room code is only a human-friendly label - a real discovery and transport layer is still needed.

6.3 Transport options (build behind one interface)

Transport	Good for	Watch out
Wi-Fi Direct	Fast peer link, real-time games	Device-specific quirks; needs permissions; test on real phones
Bluetooth	Low-bandwidth turn-based games, widest reach	Slow; pairing/permissions differ by Android version
Nearby Connections (optional)	Easy discovery and connection	Needs Google Play services - never the only option

```
interface MultiplayerTransport {
    val connectionState: StateFlow<ConnectionState>
    suspend fun connect(endpoint: Endpoint)
    suspend fun send(message: GameMessage)
    fun observeMessages(): Flow<GameMessage>
    suspend fun disconnect()
}
// Implementations: LocalPassPlayTransport, WifiTransport,
//                  BluetoothTransport, NearbyConnectionsTransport (optional)
```

The game engine only talks to this interface, so changing Bluetooth to Wi-Fi never changes a game.

6.4 Message format

```
{ "protocolVersion": 1, "sessionId": "room-123", "matchId": "match-456",
  "senderId": "player-a", "messageId": "message-789", "sequence": 12,
  "type": "PLAYER_ACTION", "gameId": "tic-tac-toe", "payload": {"position": 4} }
```

Validate every message: correct session, known sender, supported protocol version, increasing sequence number, valid payload, correct player turn, valid state. Limit packet size and message frequency. Serialise with Kotlinx Serialization.

6.5 Real-time games (Air Hockey, Tank, Sumo)

- Fixed simulation tick (1/60 s). Send **inputs**, not every animation frame.
- Host sends snapshots about 20 times a second; guests **interpolate** remote objects for smooth motion.

- Input timestamps + periodic reconciliation fix small differences.
- Do not reuse the board-game sync for physics - it needs this separate approach.

6.6 Things that will go wrong (and the answer)

Situation	What the system does
Player joins late	Host sends the latest snapshot + room settings.
Guest disconnects	Match goes DISCONNECTED; guest has a timeout to reconnect; no automatic loss.
Reconnect	Host sends last validated snapshot then missed events; duplicate sequence numbers ignored.
Host disappears	Do NOT award a win. Mark the session interrupted (host migration is a later feature).
Duplicate / delayed packets	Sequence numbers + message ids; old ones dropped.
Invalid move	Host rejects and optionally tells the guest "InvalidAction".
App in background / screen locks	Pause match; try recover on return; otherwise abandon with 0 coins.

6.7 Security and permissions

- Ask for Bluetooth / nearby-device / location permissions only when the user taps Create/Join Room, with Android-version-specific handling.
- No sensitive data in messages; authenticate room participants; encrypted connection where supported.
- Never trust a remote device's claimed score. Offline does not automatically mean secure - validate inputs and protect storage.

7. The reusable game engine (the heart)

Without a common engine every game would re-implement players, timers, scores, result screens and rewards - 28 copies of the same bugs. With it, each game only writes its *rules*.

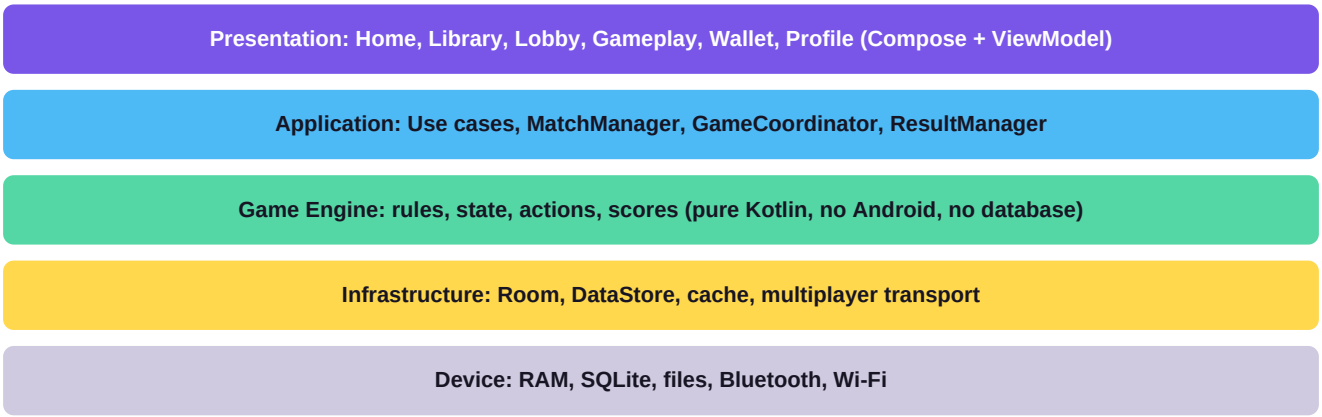
Component	Job
GameRegistry	Finds a game by unique id. New game = register it, no home-screen rewrite.
GameDefinition	Name, icon, description, category, min/max players, modes.
GameSession	One match and its players.
GameState	Current gameplay data (immutable).
GameAction	A move, tap or attack.
GameEngine / MiniGame	Validates actions and computes next state.
GameRenderer	Draws the state (Compose or Canvas).
InputController	Turns touches into actions.
PlayerManager / ScoreManager	Seats, turns, scores.
MatchManager	Start, pause, end a match.
ResultManager	Final outcome and reward hand-off.
GameSerializer	Save/restore supported matches.
GameAudioManager	Sound effects.

7.1 The contract every game implements

```
interface MiniGame<S : Any, A : Any> {
    val definition: GameDefinition
    fun createInitialState(config: GameConfig, players: List<GamePlayer>): S
    fun validateAction(state: S, action: A, playerId: String): ActionValidation
    fun reduce(state: S, action: A, playerId: String): S   // deterministic
    fun getOutcome(state: S): GameOutcome
}
sealed interface GameOutcome {
    data class Winner(val playerIds: List<String>) : GameOutcome
    data object Draw : GameOutcome
    data object Incomplete : GameOutcome
}
```

reduce() must be deterministic: same valid state + same action = same next state. This is what makes multiplayer sync, tests and save/restore possible. A game must never touch Room, DataStore or the wallet - side effects belong to the application layer.

7.2 Layers



Dependency rule: a game module may depend on the engine, shared models and UI components. The engine must never depend on a specific game (prevents circular dependencies).

7.3 Example - Tic Tac Toe through the engine

```
fun applyMove(state, action): TicTacToeState? {
    if (state.status != RUNNING) return null
    if (action.cell !in 0..8)   return null
    if (state.board[action.cell] != 0) return null
    val board = state.board.toMutableList(); board[action.cell] = state.currentPlayer
    val won  = winningLines.any { l -> l.all { board[it] == state.currentPlayer } }
    val draw = !won && board.none { it == 0 }
    return state.copy(board = board,
        currentPlayer = if (won || draw) state.currentPlayer else 3 - state.currentPlayer,
        status = when { won -> COMPLETED; draw -> DRAW; else -> RUNNING })
}
```

Flow on a tap: InputController captures tap -> engine validates -> new state -> UI renders -> Room saves result/checkpoint. In production use enums/sealed types instead of strings and validate the player id against whose turn it is.

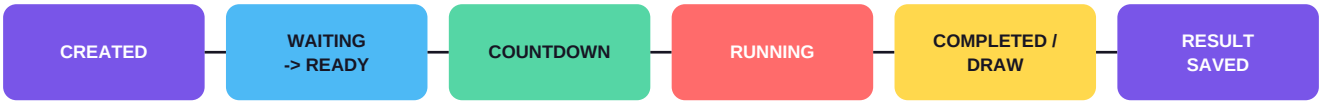
7.4 Algorithms by game family

Family	Key algorithm
Turn-based board	Receive action, check status, turn, coordinates, legality, compute state, check win/draw, advance turn, emit.
Dice (Ludo, Snakes)	Bounded random 1-6 from the trusted host; fixed route array; safe cells, captures, home entry.
Physics/arcade	Fixed step 1/60 s with accumulator: update physics, detect collisions (circle-circle, circle-rect, rect-rect), apply forces, update score, subtract step; render interpolated state. Scoring is event-based; timers use a monotonic clock.
Reaction/timing	elapsedRealtimeNanos(), random delay, record target-visible timestamp, accept tap only when active, reaction = tap - target.
Puzzle/memory	Fisher-Yates shuffle, grid scan + merge, permutation, bounded random questions, DFS maze + BFS pathfinding, sequence prefix validation.

```
private const val FIXED_STEP = 1.0 / 60.0
accumulator += frameTime
while (accumulator >= FIXED_STEP) { update(); collide(); score(); accumulator -= FIXED_STEP }
render(interpolate(alpha = accumulator / FIXED_STEP))
```

8. Match lifecycle (state machine)

Use one state machine instead of loose Boolean flags. Every game, solo or multiplayer, goes through the same states.



State	Meaning	Allowed next
CREATED	Match initialised	WAITING
WAITING	Waiting for players	READY, ABANDONED
READY	All players ready	COUNTDOWN, ABANDONED
COUNTDOWN	3-2-1	RUNNING, ABANDONED
RUNNING	Game active	PAUSED, COMPLETED, DRAW, DISCONNECTED, ABANDONED
PAUSED	Temporarily stopped	RUNNING, ABANDONED
DISCONNECTED	Device lost	RECOVERING, ABANDONED
RECOVERING	Restoring session	RUNNING, ABANDONED
COMPLETED / DRAW / ABANDONED	Terminal - no further transitions	none

Rules

- Start only after all required players are ready.
- Actions are accepted only in RUNNING; paused games reject gameplay actions.
- Disconnection does not automatically mean a loss.
- A completed match cannot be completed twice.
- Only a valid completed match produces a reward; closing the app, restarting repeatedly or disconnecting must not create coins.
- Write a unit test for every allowed and every forbidden transition via one central `canTransition(from, to)` function.

9. MiniCoins economy

MiniCoins (MC) give people a reason to play more matches and unlock cosmetics. They are local, non-transferable and non-redeemable.

9.1 Match rewards (default, tunable)

Result	Who	MC
WIN	Winner of a valid completed match	+20
DRAW	Every player in a valid draw	+10
LOSS	Other players in a completed match	+5
ABANDONED	Player exits before a valid result	0

4-player match: first place 20, others 5 each; valid draw between all 10 each. Tie for first: tied players get the draw reward, players below get the loss reward.

9.2 Single-player rewards

Achievement	Reward
Complete a level	+5 MC
Bronze / Silver / Gold target	+8 / +12 / +15 MC
Beat personal best	+5 bonus (once per record)

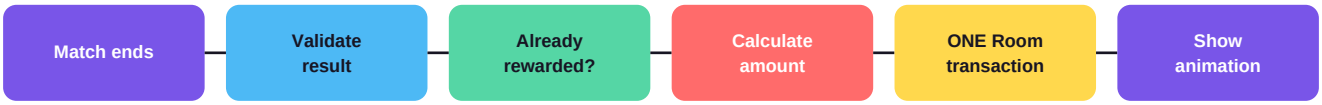
Each reward has its own unique claim id; never add them together blindly.

9.3 What coins buy (cosmetics only)

Item	Price
Basic avatar	100 MC
Rare avatar	250 MC
Special avatar	500 MC
Game background	150 MC
Premium board theme	300 MC
Profile frame	200 MC
Emote pack	100 MC

Rules: gameplay advantages are always free. No entry fees, betting, cash conversion or coin transfers - this avoids gambling-like mechanics.

9.4 Reward flow (save first, animate second)




```
suspend fun awardCoins(matchId: String, playerId: String, amount: Int) {
    database.withTransaction {
        if (coinDao.findByMatchAndPlayer(matchId, playerId) != null) return@withTransaction
        if (amount <= 0) return@withTransaction
        coinDao.insert(CoinTransactionEntity(UUID.randomUUID().toString(), matchId, playerId, amount, ...))
        walletDao.addCoins(playerId, amount)       // only after the ledger insert succeeded
    }
```

```
}
// Unique index on (matchId, playerId) -> same match can never pay twice
```

9.5 Economy database rules

- One wallet per player; every addition or deduction has a ledger record.
- Never overwrite the balance without a transaction; use Room transactions for all balance changes.
- Purchases are idempotent and can never make the balance negative.
- Store the reward-policy version used for each match.
- If the app is killed during the animation, check the ledger on restart - never replay the payout.
- The calculator only returns an amount; the repository changes the wallet.
- Offline limitation: a rooted/modified device can edit local data. We make accidents and casual cheating hard, but cannot make it tamper-proof without a server - hence cosmetic-only, non-redeemable coins and no online leaderboard claims.

Economy module layout: core/economy/{domain(model, calculator, usecase), data(dao, entity, repository), presentation(RewardViewModel, WalletViewModel, RewardAnimationController)}.

10. Data and storage

Three levels with strictly separate jobs:

Level	Holds	Notes
L1 - RAM	Live game state, ball positions, board, animation values, decoded assets	Fastest; bounded; cleared when a game closes
L2 - Disk asset cache	Images, sprites, thumbnails, audio	Bounded size; lazy loaded; WebP/vector preferred
L3 - Room	Players, matches, wallet, ledger, achievements, snapshots, history	Persistent; transactions + migrations
DataStore	Sound, vibration, theme, reduced motion, language	Small settings only

Room tables

Entity	Important fields
PlayerEntity	playerId, name, avatarId
GameEntity	gameId, name, category, enabled
MatchEntity	matchId, gameId, status, startedAt, endedAt
MatchPlayerEntity	matchId, playerId, seat, score, result
GameSnapshotEntity	matchId, stateJson, version, savedAt
CoinWalletEntity	playerId, balance
CoinTransactionEntity	transactionId, matchId, playerId, amount, reason, createdAt (unique matchId+playerId)
AchievementEntity	achievementId, playerId, unlockedAt
MatchEventEntity	matchId, sequence, eventType, payload
FavoriteGame / GameProgress / Tournament / TournamentMatch	favourites, high scores, tournament records

Database safety rules

- Primary keys, foreign keys and indices on frequently searched columns.
- Transactions for anything that must succeed together (match result + coins).
- Never do database work on the main thread; DAO functions are suspend or Flow.
- Migrations for every schema change; **never** destructive migration for player data; test every old version.
- Versioned game snapshots; validate a snapshot before restoring (corrupt save = start fresh, not crash).
- Never write every animation frame or ball position to Room - save only turn changes, pause, periodic checkpoints and results.

11. How to avoid lag and crashes

This section is the checklist to follow while coding. Treat each line as a rule.

11.1 Performance rules

- Use lazy lists/grids for the catalogue; load thumbnails first, full game assets only when a game is selected.
- Preload only the most likely next game; release audio, bitmaps, timers and loops when leaving a game.
- Keep physics and game loops **outside** Compose recomposition. Use Canvas/SurfaceView or a simple custom loop for arcade games. Compose = menus, boards, scoreboards.
- Immutable states; stable models; use remember; avoid unnecessary recomposition (derive values, hoist state).
- Fixed time step for physics; cap steps per frame; interpolate for rendering.
- Object pools for bullets, balloons, obstacles - no allocation inside the frame loop.
- No new coroutine per frame; one controlled job per session; cancel on exit.
- Bounded RAM image cache and bounded disk cache.
- Use R8 + resource shrinking, WebP/vector graphics, short compressed audio (SoundPool), system fonts.
- Frame/size budget: avoid unneeded native libraries; use App Bundle and ABI splits.

11.2 Performance targets (measure, do not assume)

Metric	Target
Cold start	Under 2 s on target devices
Cached game screen transition	Under 300 ms
Board game input response	Under 100 ms
Arcade rendering	60 FPS target
Offline functionality	100% of core features
Crash-free sessions	Above 99.5%
APK size	Under 25 MB (illustrative budget: code 8, graphics 6, audio 3, libraries 3, other 2)

11.3 Common crash/bug causes and prevention

Risk	Prevention
Double coin reward	Unique claim + single Room transaction
Invalid move	Engine validation before reduce()
Duplicate network message	Sequence number + dedupe
Wrong turn	Authoritative turn check
App killed mid-game	Versioned checkpoints
Schema change	Room migrations (tested)
Memory leak	Lifecycle-aware cleanup; clear listeners
Timer keeps running after exit	Session-scoped cancellation
Broken/missing asset	Asset validation + fallback
Unknown game id	Registry lookup returns error state, not exception

Risk	Prevention
Corrupt saved game	Snapshot validation + safe fallback
Network interruption	Reconnect + snapshot recovery
Main-thread database/IO	Always suspend/Flow on IO dispatcher
Out of memory on 2 GB phones	Bounded caches, downscaled bitmaps, release on game exit

11.4 Error handling

```
sealed interface MiniVerseError {
    data object GameNotFound; data object InvalidAction; data object SessionExpired
    data object PlayerDisconnected; data object StorageError
    data class Unknown(val message: String)
}  // map to friendly messages - never show raw exceptions
```

- Log technical errors in debug builds only; never log private data.
- Do not silently swallow exceptions.
- Never auto-retry an operation that could duplicate a reward or purchase.
- Add crash reporting only if you choose an external service and disclose it.
- Make recovery actions explicit ("Reconnect", "Restart match").

11.5 Release build settings

```
release {
    isMinifyEnabled = true
    isShrinkResources = true
    proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"), "proguard-rules.pro")
}
// Test the minified build: Room, Hilt and serialization need correct keep rules.
```

11.6 Compatibility

Area	Approach
Minimum / target Android	API 24 baseline (verify libraries); target the Play-required SDK
CPU	32-bit and 64-bit where dependencies allow
Screens	Small, medium, large, tablet; portrait default, landscape where useful
Refresh rate	Adaptive 30/60 FPS; check 90/120 Hz pacing
Memory	Bounded caches and cleanup; test on 2 GB RAM
Battery	No background services; no loops when game closed

12. Project structure and technology

```
MiniVerse/
 app/            MainActivity, MiniVerseApplication, navigation, di
 core/
   common  model  designsystem  database  datastore  cache
   gameengine  multiplayer  connectivity  economy  animation  audio  testing
 feature/
   splash onboarding home gamelibrary gameprofile multiplayerlobby
   gameplay wallet shop achievements history settings
 games/
   tic_tac_toe  connect_four  snakes_ladders  ludo  memory_match
   air_hockey  pinball  tank_battle  ...other games
 build-logic/    shared Gradle conventions
```

Module	Responsibility
app	Startup + navigation
core:model	Shared models and ids
core:database	Room entities, DAOs, migrations
core:datastore	User preferences
core:cache	Temp data + asset cache
core:gameengine	Lifecycle, contracts, sessions
core:multiplayer / connectivity	Transports, messages, sync
core:economy	Wallet, ledger, rewards
core:ui / designsystem	Shared Compose components, theme
feature:*	Screens + ViewModels
games:*	Individual game rules + renderers
core:testing	Fakes + test utilities

Do not create 28 Gradle modules on day one if it slows builds - start with logical game packages and split later.

Technology stack

Area	Technology
Language / UI	Kotlin / Jetpack Compose + Material 3
Architecture	MVVM + Clean Architecture
DI	Hilt
Database / settings	Room / DataStore
Async	Coroutines + Flow
Navigation	Navigation 3 (or one consistent supported setup)
Serialization	Kotlinx Serialization
Images	Coil
Multiplayer	Wi-Fi Direct / Bluetooth / optional Nearby Connections
Game rendering	Compose Canvas / custom rendering (SurfaceView or LibGDX only if needed)
Physics	Own lightweight engine

Area	Technology
Testing	JUnit, Turbine, Compose UI tests
Build	Gradle Kotlin DSL + version catalog

Use stable, mutually compatible library versions; do not copy outdated Gradle snippets from tutorials.

UI state pattern

```
data class HomeUiState(val isLoading: Boolean = true,
                       val games: List<GameDefinition> = emptyList(),
                       val errorMessage: String? = null)
// StateFlow in ViewModel; collect lifecycle-aware in Compose.
// State = what is shown, Event = user tapped, Effect = navigate/snackbar once.
```

Keep navigation, wallet calculations and game rules out of Composables.

13. Testing checklist

Pyramid: many unit tests (algorithms, validation, rewards), fewer integration tests (Room, multiplayer, lifecycle), few UI tests (navigation, interaction).

Area	Must pass
Game engine	Invalid moves rejected; no moves out of turn; correct win/draw detection; timers stop on pause; state restores correctly; restart creates a new session.
Coins	Win 20 / Draw 10 / Loss 5 / Abandoned 0; reopening result screen gives no second reward; two simultaneous requests do not double-credit; spending cannot go negative; wallet and ledger consistent.
Multiplayer	Two phones connect offline; invalid messages rejected; disconnect detected; reconnect restores state; duplicate packets ignored; host and guest agree on final result.
Performance	No leaks; no loops after game closes; assets released; no main-thread DB work; low battery use; smooth on low RAM.
Database	Migrations preserve data from every old version.
UI	Small screens, large font sizes, landscape, split-screen, reduced motion.

Device matrix

Device	Focus
2 GB RAM, older Android	Startup, memory, basic games
3-4 GB RAM	Gameplay and multitasking
6-8 GB RAM	Multiplayer and animations
12 GB+ RAM	Full gameplay, performance
Small display / tablet	Controls, adaptive layout
60 Hz / 90-120 Hz	Frame pacing
Real Bluetooth + Wi-Fi phones	Multiplayer - emulators are not enough

Pre-release tracker (tick each before shipping a game)

- ■ Starts and ends correctly
- ■ Invalid actions rejected
- ■ Win/draw conditions correct
- ■ No out-of-turn moves
- ■ Restart = new session
- ■ Checkpoint resume works
- ■ Match cannot award coins twice
- ■ Wallet and ledger consistent
- ■ Disconnect/reconnect works
- ■ Duplicate packets ignored
- ■ Room migrations preserve data
- ■ Leaving game releases timers, audio and memory
- ■ Small screens and large fonts work
- ■ Responsive on low-RAM devices

14. Build order - step by step

Estimate for one developer: about 4-6 months for a substantial release; multiplayer and arcade physics can extend it. Quality of the engine and multiplayer reliability matters more than game count.

Step	Weeks	Work	Milestone
1. Foundation	1-2	Android Studio project, Gradle + version catalog, Hilt, core modules, Room, DataStore, theme/design system, navigation shell, Home dashboard.	App launches without errors
2. Game engine	3-4	Contracts, GameRegistry, GameSession, MatchManager, PlayerManager, ScoreManager, state machine, serializer, caching, engine unit tests.	A dummy game runs through the engine
3. First games	5-8	Tic Tac Toe (full vertical slice first!), Connect Four, Memory Match, Snakes and Ladders, then Reaction Battle, Rock Paper Scissors, Number Merge, Whack-a-Mole.	Different game types work through one engine
4. Wallet + achievements	with 3	Player profile, MiniCoins ledger, reward transaction, history, achievements, reward animations.	Rewards survive app restarts
5. Multiplayer	9-10	Pass and play polish, host/guest sessions, discovery + pairing, action sync, reconnection, tests on physical devices.	Two real phones finish a match
6. Arcade	11-14	Pinball, Air Hockey, Finger Football, Tank Battle, Sky Jumper, Crazy Runner, Mini Sumo, Ping Pong.	Smooth, stable arcade play
7. Completion	15-20+	Remaining games, tournaments, shop, statistics, audio, haptics, animations, migration tests, profiling, accessibility.	Feature complete
8. Release	last	Size and memory optimisation, low-end phone tests, security review, signed AAB, Play Store listing.	Release candidate

The vertical-slice rule

Before building more games, make **Tic Tac Toe** work end to end: Home -> Game Details -> Setup -> Gameplay -> Result -> Room history -> MiniCoins. Once that single flow is solid, every other game reuses the same platform.

Weekly check-in suggestion: at the end of each week review (a) which milestone is done, (b) crash/lag measurements on a real low-end phone, (c) tests passing, (d) next week's three tasks.

15. Decisions to confirm before coding

Decision	Options	Recommended
1. Starting MiniCoins	0 / 50 / 100 / 250	100
2. Match rewards (Win/Draw/Loss)	20/10/5, 30/15/5, 50/20/10	20 / 10 / 5
3. Multiplayer connectivity	Adaptive (Wi-Fi + Bluetooth + optional Nearby), Wi-Fi first, Bluetooth first, Nearby first	Adaptive
4. Android package name	e.g. com.yourname.miniverse (final id cannot be changed after publishing)	Decide now

First implementation checklist

■ Create project structure and modules

■ Configure Gradle dependencies + conventions

■ Implement MiniVerse theme (palette above)

■ Build splash + onboarding

■ Build Home dashboard + Game library

■ Create GameRegistry

■ Integrate Room + DataStore

■ Implement Tic Tac Toe (vertical slice)

■ Add MiniCoins wallet + ledger

■ Add pass-and-play, then local multiplayer

Final thought: the shared engine, wallet and multiplayer layer - reusable and tested independently - are what make MiniVerse a portfolio-quality project, not just the number of games.

MiniVerse | One App. Endless Mini Adventures. | Powered by Devanshu